

Vết cạn, quay lui và cắt tỉa (Brute Force, Backtracking, Pruning)

PHỤ THUỘC A/01 cho vai trò của vết cạn trong bước 2; A/03 cho việc ước lượng kích thước không gian tìm kiếm; B/07 cho ngăn xếp và đệ quy.

TÓM TẮT

Vết cạn là lời giải luôn tồn tại và luôn đúng, nên nó là điểm xuất phát của mọi chương trong phần C. Chương này dựng nó thành một kỹ thuật đẳng hoàng: khuôn quay lui chuẩn, cách ước lượng kích thước không gian trước khi cài, và bốn nhóm cắt tỉa — cắt theo cận, cắt theo đối xứng, cắt theo ràng buộc lan truyền, và sắp thứ tự biến để thất bại sớm. Chương cũng nói về gặp nhau ở giữa, kỹ thuật biến 2^n thành $2^{n/2}$, và về ranh giới quan trọng: khi nào vết cạn có cắt tỉa *chính là* lời giải đúng chứ không phải giải pháp tạm. Nếu chỉ nhớ một điều: đừng hỏi “làm sao tránh vết cạn”, hãy hỏi “vết cạn của tôi đang lãng phí ở đâu” — câu thứ hai dẫn thẳng tới mọi kỹ thuật của bảy chương còn lại.

KIẾN THỨC CẦN CÓ TRƯỚC

- **không gian tìm kiếm** — tập mọi cấu hình mà thuật toán có thể phải xét; kích thước của nó quyết định vết cạn có khả thi hay không.
- **quay lui (backtracking)** — xây lời giải từng bước; khi một bước dẫn tới ngõ cụt thì huỷ bước đó và thử lựa chọn khác. Về bản chất là duyệt sâu trên cây các lời giải bộ phận.
- **cắt tỉa (pruning)** — loại bỏ cả một nhánh của cây tìm kiếm khi chứng minh được nhánh đó không chứa lời giải tốt hơn cái đã có.
- **nhánh cận (branch and bound)** — quay lui có kèm chặn: nếu cận tốt nhất có thể của một nhánh còn tệ hơn lời giải đã tìm được thì bỏ nhánh.
- **gặp nhau ở giữa (meet in the middle)** — chia bài toán thành hai nửa độc lập, liệt kê đủ mỗi nửa ($2^{n/2}$), rồi ghép hai danh sách bằng sắp xếp hoặc bảng băm.
- **phá đối xứng** — loại bỏ những cấu hình chỉ khác nhau bởi một phép hoán vị hay phản chiếu vô nghĩa, để không đếm và không duyệt trùng.

CÂU HỎI MỞ ĐẦU

- Q1. Vết cạn luôn đúng và luôn chậm. Vậy vì sao chương đầu tiên của phần thiết kế lại dành cho nó?
- Q2. Bài toán tám quân hậu có $64^8 \approx 2,8 \cdot 10^{14}$ cách đặt thô, nhưng chương trình quay lui giải trong tích tắc. Ba tầng cắt giảm nào đã xảy ra?
- Q3. Cắt tỉa không đối độ phức tạp trường hợp xấu nhất. Vậy tại sao nó lại tạo khác biệt lớn tới vậy trong thực tế?
- Q4. Kỹ thuật gặp nhau ở giữa đưa 2^n xuống $2^{n/2}$ — vẫn là hàm mũ. Vì sao điều đó vẫn thay đổi được kết cục của bài toán?
- Q5. Khi nào một lời giải vết cạn có cắt tỉa là câu trả lời *đúng* cho bài toán, chứ không phải là giải pháp tạm bợ?

Phần A kết thúc bằng một lời khuyên lặp lại nhiều lần: khi bí, hãy viết ra lời giải vét cạn trước. Chương này giữ đúng lời khuyên đó nhưng đi xa hơn — nó chỉ ra rằng vét cạn không chỉ là bước đệm mà là một kỹ thuật có kỹ thuật riêng, và trong một số bài, vét cạn được cắt tỉa tốt *chính là* lời giải tốt nhất mà loài người biết.

Điều này đáng nhấn mạnh vì người mới thường coi vét cạn là thứ đáng xấu hổ. Thực tế thì các bộ giải SAT hiện đại — công cụ giải những bài logic công nghiệp với hàng triệu biến — về bản chất là quay lui có học từ thất bại. Các máy chơi cờ mạnh nhất trong nhiều thập kỷ là tìm kiếm có cắt tỉa. Đó không phải giải pháp tạm; đó là trạng thái nghệ thuật cho những bài toán mà không ai biết lời giải đa thức.

1.1 Ước lượng trước khi cài

Việc đầu tiên, và nó tốn ba mươi giây: *ước lượng kích thước không gian tìm kiếm*. Ba dạng thường gặp và cách nhận ra chúng trên đề bài:

Tập con — 2^n cấu hình. Dấu hiệu: mỗi phần tử chỉ có hai lựa chọn, chọn hoặc không. Khả thi tới $n \approx 25$.

Hoán vị — $n!$ cấu hình. Dấu hiệu: sắp thứ tự, gán một-một. Khả thi tới $n \approx 11$.

Gán k giá trị cho n biến — k^n . Dấu hiệu: tô màu, điền số. Khả thi khi k^n dưới cỡ 10^8 .

Bảng ở chương A/03 biến các con số này thành quyết định. Nhưng có một điều quan trọng hơn con số thô: ước lượng đó là *ước lượng trước khi cắt tỉa*. Với cắt tỉa tốt, số nhánh thực sự được duyệt có thể nhỏ hơn hàng chục bậc — và đó chính là lý do bài tám quân hậu giải được. Vì vậy quy trình đúng là: ước lượng thô để biết mình đang ở đâu, rồi hỏi “ràng buộc nào của đề cho phép tôi cắt sớm”.

1.2 Khuôn quay lui và bốn tầng cắt

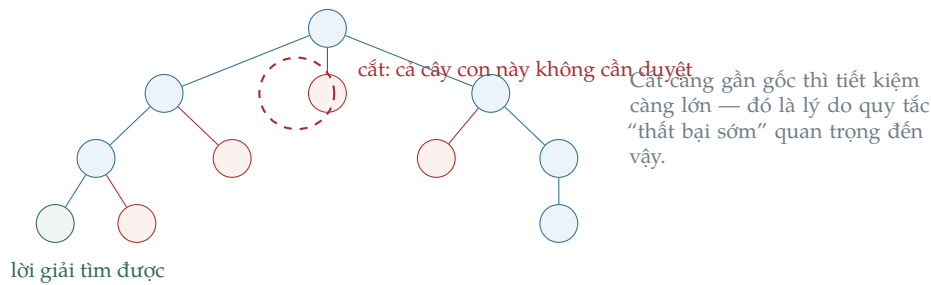
Mọi thuật toán quay lui đều có cùng khuôn: chọn biến tiếp theo, thử từng giá trị khả dĩ, kiểm tra tính hợp lệ, đệ quy, rồi huỷ lựa chọn. Phần khác nhau — và phần tạo ra toàn bộ hiệu năng — nằm ở bốn chỗ cắt.

Tầng 1 — cắt theo ràng buộc ngay khi đặt. Đừng đặt xong cả cấu hình rồi mới kiểm tra. Với bài tám quân hậu, ngay khi đặt quân thứ hai đã kiểm tra xung đột với quân thứ nhất; nhánh nào vi phạm thì cắt luôn. Chỉ tầng này thôi đã đưa 64^8 xuống cỡ vài nghìn nút.

Tầng 2 — phá đối xứng. Nhiều cấu hình chỉ khác nhau bởi một phép hoán vị vô nghĩa. Nếu bài không phân biệt thứ tự các phần tử được chọn, hãy ép chỉ số tăng dần; nếu bàn cờ đối xứng, hãy cố định quân đầu tiên ở nửa trái. Cắt này thường cho hệ số 2 tới $n!$ tùy bài, và nó cũng chữa luôn lỗi đếm trùng — một lỗi rất hay gặp trong các bài đếm.

Tầng 3 — cắt theo cận (nhánh cận). Nếu đang tìm cực tiểu và đã có lời giải giá B , thì với một nhánh mà ngay cả trong trường hợp tốt nhất vẫn không thể dưới B , hãy bỏ nhánh. Chất lượng của cắt này phụ thuộc vào chất lượng của hàm cận: cận càng chặt thì cắt càng nhiều, nhưng tính cận cũng tốn. Đây là một sự đánh đổi phải cân bằng bằng thực nghiệm.

Tầng 4 — thứ tự thử. Hai quy tắc kinh nghiệm mạnh. Thứ nhất, *chọn biến bị ràng buộc nhiều nhất trước* — ô sudoku còn ít khả năng nhất, quân cờ có ít nước đi nhất — vì như vậy thất bại xảy ra sớm, gần gốc, cắt được cây to. Thứ hai, *thử giá trị hứa hẹn nhất trước* khi tìm cực trị, vì tìm được lời giải tốt sớm làm cận ở tầng 3 chặt hơn cho toàn bộ phần còn lại.



Hình 1.1. Cây quay lui với các nhánh bị cắt (đỏ). Điều đáng nhớ: giá trị của một lần cắt tỉ lệ với kích thước cây con bị loại, mà kích thước đó giảm theo hàm mũ theo độ sâu. Vì vậy một cắt ở tầng 2 đáng giá hơn hàng nghìn cắt ở tầng 10, và đó là toàn bộ lý do của quy tắc “chọn biến bị ràng buộc nhiều nhất trước”.

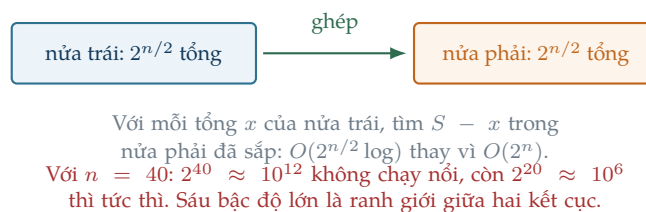
CẦN NHỚ

Cắt tia *không* cải thiện độ phức tạp trường hợp xấu nhất — bài toán vẫn là hàm mũ. Nó chỉ thay đổi trường hợp trung bình trên dữ liệu thực tế, nhưng thay đổi đó thường là vài bậc độ lớn, đủ để biến “không chạy nổi” thành “xong trong một giây”. Đây là một trong số ít chỗ trong quyển sách này mà lý thuyết và thực hành nói hai câu chuyện khác nhau, và cả hai đều đúng.

1.3 Gặp nhau ở giữa: đổi bộ nhớ lấy thời gian

Khi n khoảng 40, $2^n \approx 10^{12}$ là quá lớn, nhưng $2^{n/2} \approx 10^6$ thì rất thoải mái. Kỹ thuật gặp nhau ở giữa khai thác đúng khe đó: chia tập thành hai nửa, liệt kê đủ mọi tập con của mỗi nửa, rồi ghép hai danh sách.

Ví dụ chuẩn: cho $n \leq 40$ số, hỏi có tập con nào có tổng bằng S không. Liệt kê mọi tổng của nửa đầu (2^{20} giá trị) và mọi tổng của nửa sau; sắp xếp danh sách thứ hai, rồi với mỗi tổng x của nửa đầu, tìm nhị phân $S - x$. Chi phí $O(2^{n/2} \cdot n)$ thời gian và $O(2^{n/2})$ bộ nhớ.



Hình 1.2. Gặp nhau ở giữa. Điều cần nhận ra trên đề bài chính là giới hạn: n trong khoảng 30–45 nằm đúng giữa hai vùng — quá lớn cho 2^n , quá nhỏ để đòi hỏi lời giải đa thức — nên nó gần như luôn là lời mời dùng kỹ thuật này.

Điều đáng nhớ về mặt tư duy không phải bản thân kỹ thuật mà là *dấu hiệu* nhận biết: giới hạn n khoảng 30–45 gần như luôn là lời mời gặp nhau ở giữa, vì nó nằm đúng giữa hai vùng — quá lớn cho 2^n , quá nhỏ để đòi hỏi lời giải đa thức. Đây là ví dụ rõ nhất cho luận điểm ở chương A/03: giới hạn trên đề là một nửa gợi ý.

Điều kiện áp dụng cũng cần nói rõ: hai nửa phải *độc lập* và phải *ghép được bằng một phép tra cứu*. Nếu quyết định ở nửa sau phụ thuộc chi tiết vào cấu hình cụ thể của nửa đầu chứ không chỉ vào một giá trị tổng hợp, thì kỹ thuật này không áp dụng — và lúc đó bạn đang ở địa hạt của quy hoạch động (C/16).

1.4 Khi vết cạn là câu trả lời đúng

Có ba tình huống mà vết cạn có cắt tia không phải giải pháp tạm mà là lời giải tốt nhất đã biết.

Thứ nhất: bài toán NP-khó với dữ liệu nhỏ. Không ai biết thuật toán đa thức, và n đủ nhỏ. Lúc đó câu hỏi không còn là “có tránh được hàm mũ không” mà là “hằng số của hàm mũ nhỏ tới đâu”. Chương D/23 bàn kỹ.

Thứ hai: bài toán công nghiệp có cấu trúc. Các bài logic, lập lịch và kiểm chứng phần cứng trong thực tế có rất nhiều cấu trúc ẩn, và các bộ giải hiện đại khai thác được. Một bộ giải SAT hiện nay có thể xử lý những bài hàng triệu biến — không phải vì nó tránh được hàm mũ mà vì nó *học từ mỗi lần thất bại*: khi một nhánh mâu thuẫn, nó suy ra một mệnh đề mới ngăn cả một lớp cấu hình tương tự. Đây là bước tiến khái niệm quan trọng nhất của quay lui trong ba chục năm qua.

Thứ ba: khi bạn cần một lời giải chắc chắn đúng để đối chiếu. Trong quy trình stress test ở chương E/25, lời giải vết cạn đóng vai trò trọng tài. Nó chậm nhưng chạy trên dữ liệu nhỏ, và nó là thứ duy nhất bạn tin tuyệt đối.

1.5 Gốc rễ: từ trò chơi ô chữ tới bộ giải công nghiệp

Quay lui là kỹ thuật cổ, và nó có tên trước khi có máy tính hiện đại: nhà toán học D. H. Lehmer đặt ra thuật ngữ *backtrack* trong thập niên 1950 khi mô tả cách giải các bài toán tổ hợp bằng máy. Đến năm 1965, Golomb và Baumert viết một khảo sát hệ thống hoá kỹ thuật này, và nhiều mẹo trong mục 2 — phá đối xứng, chọn biên ràng buộc nhiều nhất — đã có mặt ở đó.

Có một đóng góp của Knuth đáng nhắc riêng vì nó rất thực dụng và ít người biết. Năm 1975, ông công bố một cách *ước lượng* kích thước cây quay lui mà không phải duyệt hết: chạy thử một đường ngẫu nhiên từ gốc xuống lá, mỗi bước nhân số nhánh khả dĩ vào một tích, lặp lại vài trăm lần rồi lấy trung bình. Kết quả là ước lượng không chệch cho tổng số nút. Ý nghĩa thực hành rất lớn: trước khi bỏ hai giờ chờ một chương trình quay lui, bạn tốn vài giây để biết nó sẽ chạy vài giây hay vài năm. Đây là một ví dụ đẹp cho tinh thần “đo trước khi làm” ở chương A/02.

Nhánh nhánh cạn đến từ nghiên cứu vận trù học: Land và Doig đề xuất nó năm 1960 cho quy hoạch nguyên, tức là để giải các bài tối ưu công nghiệp có biến nguyên. Nhánh SAT bắt đầu năm 1962 với thuật toán DPLL — vẫn là quay lui, cộng thêm hai quy tắc lan truyền ràng buộc. Bước nhảy vọt thật sự đến vào cuối thập niên 1990 với ý tưởng *học mệnh đề từ xung đột*: mỗi lần bế tắc, bộ giải phân tích nguyên nhân và ghi lại một ràng buộc mới. Chính bước này biến SAT từ bài toán học thuật thành công cụ công nghiệp, dùng để kiểm chứng thiết kế vi mạch và phân tích chương trình.

Cuối cùng, một liên hệ đáng nhớ với miền khác: trong thị giác máy tính và robot, khi bài toán ước lượng có quá nhiều tổ hợp dữ liệu ngoại lai, người ta không vết cạn mà *lấy mẫu ngẫu nhiên* — kỹ thuật RANSAC. Đó là một câu trả lời khác cho cùng câu hỏi “không gian quá lớn thì làm gì”, và chương C/20 sẽ khai thác chính hướng đó.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Thập niên 1950 — Lehmer	Giải các bài toán tổ hợp bằng máy tính đời đầu.	Thuật ngữ <i>backtrack</i> ; ý tưởng huỷ lựa chọn và thử nhánh khác được đặt tên.
1960 — Land & Doig	Bài tối ưu công nghiệp có biến nguyên, không giải được bằng quy hoạch tuyến tính.	Nhánh cạn: dùng cạn nói lòng để cắt nhánh; nền của các bộ giải quy hoạch nguyên ngày nay.

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó (tiếp)
1962 — Davis, Putnam, Logemann, Loveland	Chứng minh tự động các mệnh đề logic.	DPLL: quay lui + lan truyền ràng buộc; khung sườn vẫn dùng tới nay.
1965 — Golomb & Baumert	Nhiều người tự phát minh lại cùng các mẹo quay lui.	Khảo sát hệ thống; phá đối xứng, chọn biến ràng buộc nhất, kiểm tra sớm.
1975 — Knuth	Không biết một chương trình quay lui sẽ chạy vài giây hay vài năm.	Ước lượng kích thước cây bằng cách lấy mẫu đường ngẫu nhiên — vài giây để biết có nên chạy hay không.
Cuối thập niên 1990 — học mệnh đề từ xung đột	Bộ giải SAT bế tắc rồi lặp lại đúng sai lầm ở nhánh khác.	Học từ mỗi thất bại thành một ràng buộc mới; biến SAT thành công cụ công nghiệp cho kiểm chứng vi mạch và phân tích chương trình.

1.6 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Cờ vua và trò chơi	Tìm kiếm cây trò chơi với cắt tỉa alpha-beta	Alpha-beta <i>chính là</i> nhánh cận cho trò chơi hai người: cận trên và cận dưới thay cho một cận. Cùng quy tắc thứ tự: thử nước hứa hẹn trước để cắt được nhiều hơn.
Kiểm chứng phần cứng, phân tích chương trình	Bộ giải SAT/SMT	Cùng khung quay lui, cộng học từ xung đột. Bài học chuyển được: khi bế tắc, đừng chỉ lùi một bước — hãy hỏi <i>nguyên nhân</i> bế tắc và ghi lại nó để không lặp lại.
Vận trù học	Quy hoạch nguyên, nhánh cận với nới lỏng tuyến tính	Cùng cấu trúc; cận đến từ việc bỏ ràng buộc nguyên và giải bài liên tục. Ý tưởng chung rất mạnh: cận tốt thường đến từ một <i>phiên bản dễ hơn</i> của chính bài toán — đúng heuristic “bỏ bớt ràng buộc” ở A/01.
Trí tuệ nhân tạo cổ điển	Thoả mãn ràng buộc, lan truyền ràng buộc, cung nhất quán	Tầng cắt thứ nhất và thứ tư của mục 2 chính là các kỹ thuật ở đó dưới tên khác. Cùng phát hiện: chi phí lan truyền ràng buộc thường rẻ hơn nhiều so với chi phí duyệt nhánh chết.
Thị giác máy tính, SLAM	RANSAC: lấy mẫu ngẫu nhiên thay vì duyệt mọi tổ hợp	Câu trả lời khác cho cùng vấn đề “không gian quá lớn”. Khác biệt cốt lõi: RANSAC từ bỏ tính đầy đủ để đổi lấy tốc độ, còn quay lui giữ tính đầy đủ và cắt những nhánh <i>chứng minh được</i> là vô ích.
Hoá học, sinh học tính toán	Tìm cấu hình phân tử, gấp protein	Không gian cấu hình khổng lồ, cắt theo năng lượng chính là nhánh cận. Khác: hàm mục tiêu ở đó chỉ là mô hình xấp xỉ, nên “lời giải tối ưu” của bài toán không chắc là lời giải đúng của thực tại.
Mật mã	Tấn công vết cặn khoá, đánh đổi thời gian-bộ nhớ	Gặp nhau ở giữa là một kỹ thuật thám mã kinh điển; nó chính là lý do một số sơ đồ mã hoá kép không an toàn gấp đôi như trực giác gợi ý.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì nó là điểm xuất phát chung của cả bảy chương còn lại. Mọi kỹ thuật sau này đều được sinh ra bằng cách hỏi “vết cặn lãng phí ở đâu”: tính lại thứ đã tính dẫn tới quy hoạch động, duyệt nhánh chắc chắn tẻ dẫn tới tham lam và cắt tỉa, nhìn sai dạng bài dẫn tới mô hình hoá lại. Ngoài

ra vết cặn còn là trọng tài cho stress test, và trong nhiều bài NP-khó nó là lời giải tốt nhất đã biết.

- A2. Tầng một: mỗi cột chỉ đặt một quân, đưa 64^8 xuống $8^8 \approx 1,7 \cdot 10^7$. Tầng hai: kiểm tra xung đột hàng và đường chéo *ngay khi đặt* thay vì đặt xong mới kiểm, cắt phần lớn nhánh ở gần gốc và đưa số nút xuống cỡ vài nghìn. Tầng ba: phá đối xứng, ví dụ chỉ đặt quân đầu ở nửa trái rồi nhân đôi kết quả. Điều đáng rút ra là tầng hai — kiểm tra sớm — đóng góp lớn nhất, đúng như hình 1.1 chỉ ra.
- A3. Vì trường hợp xấu nhất hiếm khi là trường hợp bạn gặp. Dữ liệu thực tế có cấu trúc: các ràng buộc mâu thuẫn nhau sớm, các cặn chặn chặt, và phần lớn cây tìm kiếm chết ở gần gốc. Cắt tỉa khai thác đúng cấu trúc đó. Nói theo ngôn ngữ chương A/03: độ phức tạp tiệm cận mô tả trường hợp xấu nhất của *mọi* dữ liệu, còn cắt tỉa cải thiện phân phối thực tế — hai câu chuyện khác nhau và cả hai đều đúng.
- A4. Vì hàm mũ với số mũ bằng nửa là một hàm hoàn toàn khác về mặt thực tiễn: 2^{40} là 10^{12} , còn 2^{20} là 10^6 . Sáu bậc độ lớn là ranh giới giữa “không chạy nổi” và “xong tức thì”. Cái giá là bộ nhớ $O(2^{n/2})$, tức là ta đổi thời gian lấy bộ nhớ — một đánh đổi rất đáng khi bộ nhớ còn dư. Dấu hiệu nhận biết trên đề: n khoảng 30–45.
- A5. Ba tình huống. Khi bài toán NP-khó và n nhỏ — không tồn tại lời giải đa thức đã biết, nên câu hỏi chỉ còn là hằng số của hàm mũ. Khi bài toán công nghiệp có cấu trúc mà bộ giải khai thác được — SAT và quy hoạch nguyên giải những bài hàng triệu biến bằng đúng cách này. Và khi bạn cần một lời giải chắc chắn đúng làm trọng tài cho stress test (E/25). Trong cả ba, vết cặn không phải bước lùi mà là lựa chọn có cân nhắc.

NÓI LẠI THẬT ĐƠN GIẢN

Vết cặn là cách giải mà ai cũng nghĩ ra: thử hết mọi khả năng. Vấn đề duy nhất là “mọi khả năng” thường nhiều tới mức không sống nổi để thử hết.

Nhưng hãy để ý cách một người thật sự đi tìm chìa khoá trong nhà. Họ không mở lần lượt từng ngăn kéo trong toàn bộ căn nhà. Họ nghĩ: “chìa khoá không thể ở trong tủ lạnh” — và loại cả căn bếp bằng một câu. Đó chính là cắt tỉa: bỏ cả một vùng vì *chắc chắn* không có gì trong đó, chứ không phải vì lười tìm.

Ba mẹo mà người tìm chìa khoá dùng cũng chính là ba mẹo trong chương này. Một, kiểm tra sớm: vừa mở ngăn kéo đã thấy nó rỗng thì đóng lại ngay, đừng lục từng góc. Hai, đừng tìm hai lần cùng một chỗ chỉ vì bạn đi vào phòng theo hướng khác — đó là phá đối xứng. Ba, tìm chỗ khả nghi nhất trước; nếu tìm thấy sớm thì bạn khỏi phải lục phần còn lại.

Có một mẹo thứ tư rất hay dùng khi số khả năng quá lớn: chia đôi bài toán. Nếu bạn cần tìm hai món ghép lại thành một tổng cho trước, đừng thử mọi cặp. Hãy liệt kê mọi khả năng của nửa trái ra một danh sách, rồi với mỗi khả năng của nửa phải, tra xem cái bù của nó có trong danh sách không. Số việc từ “nhân” thành “cộng”, và đó là toàn bộ ý của ghép nhau ở giữa.

Cuối cùng, đừng coi thường vết cặn. Có những bài mà cả thế giới không biết cách nào tốt hơn ngoài thử hết một cách thông minh — và các phần mềm kiểm tra thiết kế vi mạch ngày nay chính là những cỗ máy vết cặn rất giỏi cắt tỉa và biết rút kinh nghiệm sau mỗi lần đâm vào ngõ cụt.

Một câu: Đừng hỏi làm sao tránh vết cặn; hãy hỏi vết cặn của tôi đang phí công ở đâu.

Chỗ không rút gọn được: Không có cách nào biết trước một hàm cặn có đủ chặt để cắt hiệu quả hay không; phải cài và đo.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Vai trò của vết cặn	Điểm xuất phát của mọi kỹ thuật ở phần C; trọng tài cho stress test; và là lời giải tốt nhất đã biết cho nhiều bài NP-khó.
••• Ba dạng không gian và ngưỡng	Tập con 2^n (tới $n \approx 25$); hoán vị $n!$ (tới $n \approx 11$); gán k giá trị k^n . Ước lượng trước khi cài.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Bốn tầng cắt	Kiểm tra ràng buộc ngay khi đặt; phá đối xứng; cắt theo cặn (nhánh cặn); chọn thứ tự biến và giá trị để thất bại sớm.
●●● Vì sao “thất bại sớm” quan trọng	Giá trị của một lần cắt tỉ lệ với kích thước cây con bị loại, mà cây con giảm theo hàm mũ theo độ sâu — cắt gần gốc đáng giá gấp bội.
●●● Cắt tỉa và độ phức tạp	Không cải thiện trường hợp xấu nhất; chỉ cải thiện phân phối thực tế — nhưng thường vài bậc độ lớn.
●●● Gặp nhau ở giữa	Chia đôi, liệt kê $2^{n/2}$ mỗi nửa, ghép bằng sắp xếp hoặc băm. Dấu hiệu: n khoảng 30–45. Đổi bộ nhớ lấy thời gian.
●● Điều kiện của gặp nhau ở giữa	Hai nửa độc lập và ghép được qua một giá trị tổng hợp. Nếu nửa sau cần biết chi tiết cấu hình nửa đầu thì phải chuyển sang quy hoạch động.
●● Hai quy tắc thứ tự	Chọn biến bị ràng buộc nhiều nhất trước; thử giá trị hứa hẹn nhất trước (để cặn sớm chặt hơn).
●● Ước lượng cây quay lui của Knuth	Chạy vài trăm đường ngẫu nhiên từ gốc, nhân số nhánh dọc đường, lấy trung bình — ước lượng không chệch cho tổng số nút.
●● Học từ xung đột	Khi bế tắc, phân tích nguyên nhân và ghi lại thành một ràng buộc mới để không lặp lại ở nhánh khác. Bước tiến lớn nhất của quay lui hiện đại.
●● Alpha-beta là gì	Nhánh cặn cho trò chơi hai người: duy trì đồng thời cặn trên và cặn dưới. Cùng quy tắc thứ tự nước đi.
● Nguồn của một cặn tốt	Thường là lời giải của một <i>phiên bản dễ hơn</i> của chính bài toán (bỏ bớt ràng buộc) — đúng heuristic của A/01.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Làm sao biết chương trình quay lui của tôi sẽ chạy bao lâu trước khi chạy?

| Dùng phép ước lượng của Knuth ở mục 5: chạy vài trăm lần một đường đi ngẫu nhiên từ gốc xuống, tại mỗi nút nhân vào tích số nhánh khả dĩ, và lấy trung bình các tích. Con số đó là ước lượng không chệch cho tổng số nút của cây. Cách này tốn vài giây và cho bạn biết ngay thứ tự độ lớn — rất đáng làm trước khi ngồi chờ một chương trình có thể chạy hàng giờ.

2. Quay lui nên viết đệ quy hay dùng ngăn xếp tường minh?

| Đệ quy trong hầu hết trường hợp: nó ngắn hơn, khớp với cấu trúc bài toán, và việc “huỷ lựa chọn” xảy ra tự nhiên khi hàm trả về. Chuyển sang ngăn xếp tường minh chỉ khi độ sâu quá lớn gây tràn, hoặc khi bạn cần tạm dừng và tiếp tục việc tìm kiếm — ví dụ tìm kiếm có giới hạn thời gian trong một hệ thống thật.

3. Tôi nên phá đối xứng thế nào cho đúng, tránh làm mất nghiệm?

| Nguyên tắc: đặt một *thứ tự chuẩn tắc* và chỉ duyệt các cấu hình chuẩn tắc. Ví dụ với tập con thì ép chỉ số tăng dần; với việc chia n phần tử vào các nhóm không phân biệt thì ép phần tử đầu tiên của mỗi nhóm mới phải tăng dần. Cách kiểm tra an toàn: đếm số nghiệm với và không có phá đối xứng trên n nhỏ, rồi đối chiếu với con số kỳ vọng (có thể phải nhân lại hệ số đối xứng).

4. Nhánh cặn cần hàm cặn, mà hàm cặn lấy ở đâu ra?

| Gần như luôn từ một *phiên bản nói lỏng* của bài toán: bỏ bớt một ràng buộc rồi giải bài dễ hơn. Với bài ba lô, cặn là lời giải của phiên bản cho phép lấy phần lẻ của món đồ — giải bằng tham lam. Với bài người bán hàng, cặn có thể là tổng các cạnh nhẹ nhất tại mỗi đỉnh, hoặc trọng số cây khung nhỏ nhất. Đây là ứng dụng trực tiếp của heuristic “bỏ bớt ràng buộc” ở A/01: bài dễ hơn không chỉ giúp bạn hiểu bài, nó còn cho bạn một cặn.

5. Khi nào quay lui thua quy hoạch động và ngược lại?

| Quy hoạch động thắng khi các nhánh khác nhau dẫn tới *cùng một trạng thái* và trạng thái đó mô tả được gọn — lúc đó lưu lại kết quả tránh tính lại. Quay lui thắng khi không gian trạng thái quá lớn hoặc không nên được thành ít chiều, nhưng ràng buộc lại chặt tới mức phần lớn nhánh chết sớm. Dấu hiệu thực dụng: nếu bạn viết được một khoá trạng thái ngắn thì hãy nghĩ tới quy hoạch động (C/16); nếu khoá đó phải chứa cả cấu hình đã chọn thì quay lui có cắt tỉa thường là hướng đúng.

6. Có nên dùng ghi nhớ cho quay lui không?

| Có, và ranh giới giữa hai kỹ thuật mờ hơn người ta tưởng. Nếu nhiều nhánh dẫn tới cùng trạng thái, hãy lưu kết quả — lúc đó bạn đã đi từ quay lui sang quy hoạch động mà không đổi cấu trúc code. Điều cần cẩn thận là khoá trạng thái: nó phải *đủ* để xác định phần còn lại của bài toán. Khoá thiếu thông tin cho kết quả sai một cách im lặng, và đây là lỗi phổ biến nhất khi thêm ghi nhớ vào một hàm đệ quy đã chạy đúng.

7. Bộ giải SAT thực sự làm gì khác so với quay lui trong sách giáo khoa?

| Ba thứ, và cả ba đều là ý tưởng chuyển được. Một, lan truyền ràng buộc rất mạnh: mỗi lần gán một biến, suy ra ngay mọi hệ quả bắt buộc. Hai, học mệnh đề từ xung đột: khi bế tắc, phân tích *nguyên nhân* và ghi lại một ràng buộc mới, nhờ đó không lặp lại sai lầm ở nhánh khác. Ba, quay lui không tuần tự: thay vì lùi một bước, nhảy thẳng về mức thực sự gây ra mâu thuẫn. Ý tưởng thứ hai là thứ đáng mang theo nhất, kể cả khi bạn tự viết quay lui cho một bài cụ thể.

I Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Sinh mọi tập con và mọi hoán vị của n phần tử	CSES “Creating Strings”, “Apple Division”[1]	Viết cả bản đệ quy lẫn bản dùng mặt nạ bit; hiểu rõ cả hai.
● Bài toán tám quân hậu	CSES “Chessboard and Queens”[1]	Đếm số nút cây với và không có kiểm tra sớm, để thấy hiệu quả bằng số liệu.
●● Tô màu đồ thị bằng k màu	Bài kinh điển	Chọn đỉnh có nhiều màu bị cấm nhất trước — quy tắc “thất bại sớm”.
●● Giải sudoku	Bài kinh điển	Kết hợp lan truyền ràng buộc với chọn ô ít khả năng nhất; so thời gian với bản không có hai mẹo đó.
●● Tập con có tổng bằng S với $n \leq 40$	Bài kinh điển	Gặp nhau ở giữa; dấu hiệu là chính giới hạn n .
●● Chia n món đồ thành hai nhóm sao cho chênh lệch nhỏ nhất	CSES “Apple Division”[1]	$n \leq 20$ nên duyệt tập con là đủ; sau đó thử lại với $n = 40$ bằng gặp nhau ở giữa.
●●● Bài toán người bán hàng với $n \leq 20$	Bài kinh điển	Quy hoạch động bitmask (C/16); rồi thử nhánh cận và so hai cách trên cùng dữ liệu.
●●● Đếm số cách đặt n quân hậu, có phá đối xứng	Bài kinh điển	Kiểm chứng bằng cách so với dãy kết quả đã biết cho n nhỏ.
●●● Ước lượng kích thước cây quay lui bằng phương pháp Knuth	Bài tập thực nghiệm	Áp dụng cho bài n quân hậu; so ước lượng với số nút thật.
●●● Cài một bộ giải SAT đơn giản kiểu DPLL	Bài tập cài đặt	Thêm lan truyền đơn vị, rồi đo lại; đây là cách rẻ nhất để hiểu vì sao lan truyền ràng buộc mạnh đến vậy.
●●● Bài toán phủ chính xác (xếp hình pentomino hoặc tương tự)	Bài kinh điển	Cấu trúc “liên kết nhảy” giúp huỷ và khôi phục lựa chọn trong $O(1)$; một bài tập cài đặt đẹp.

Kết chương và đọc tiếp

Chương này mở phần C bằng thứ mà mọi lời giải đều bắt đầu. Ba điều đáng mang theo: ước lượng không gian trước khi cài; bốn tầng cắt, trong đó “thất bại sớm” đáng giá nhất vì cắt gần gốc; và ý thức rằng vết cạn có cắt tỉa đôi khi *chính là* lời giải đúng, không phải bước lùi.

Bảy chương còn lại của phần C đều trả lời câu hỏi “vết cạn của tôi lãng phí ở đâu”. Câu trả lời đầu tiên: nếu bài toán chia được thành các phần *độc lập* thì chi phí không cộng lại mà sụp xuống theo hệ thức truy hồi — đó là chia để trị (C/14), và nó cũng là nơi ta gặp lại một ý tưởng của chương B/09: ngẫu nhiên hoá để không có trường hợp xấu.

Nguồn

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).